

Camera: Deep-Research Architecture for a Universal Computational RAW Android Camera

Product thesis and hard platform limits

Your idea is technically ambitious but coherent: build “**Camera**” as a Camera2-first computational-photography application whose primary photographic asset is sensor-domain RAW, whose processing pipeline is owned by the app, whose exposed lenses are explicitly tested rather than blindly enumerated, and whose behavior is configurable per physical lens. Android's Camera2 API is the correct foundation because it is the platform's low-level camera API and exposes per-capture sensor/lens controls, RAW output, logical/physical multicamera information, manual sensor control, and stream configuration details that higher-level camera APIs deliberately abstract. ¹

I interpret your reference to “sanpcam/code aurora” as the Snapdragon/CodeAurora `org.codeaurora.snapcam` style of low-level Android camera application. The important lesson to take from that lineage is **low-level control and device-specific capability discovery**, not package-name tricks or copied proprietary/vendor code. For a universal application, capabilities have to be determined from Camera2/NDK metadata and verified session configurations, because Android explicitly makes physical-camera and stream-combination support device-dependent. ²

The single biggest architectural principle should be:

RAW master first; everything else is a reversible interpretation of that master whenever mathematically possible.

Android's DNG support is explicitly intended for minimally preprocessed camera-sensor data and can construct DNG metadata from CameraCharacteristics and CaptureResult. It can even write application-generated Bayer-type raw data, which makes a fused computational RAW master practical if you preserve a valid Bayer grid and proper calibration metadata. ³

There are, however, several things the app must **not pretend Android can guarantee**:

Your goal	What “Camera” can realistically guarantee
Access every rear/front auxiliary sensor	Only cameras/physical sub-cameras that the framework/HAL exposes to the application. Camera IDs restricted as system cameras remain privileged. ⁴
Hide useless/raw/depth IDs	Yes. Build a tested usable-lens database and show only candidates that successfully create the required sessions.
Completely bypass Android/OEM camera stack	No. Sensor data still travels through sensor firmware, the camera HAL, CameraService and Camera2/NDK. RAW lets you bypass much of the conventional ISP rendering pipeline, not the entire camera platform. Google's original HDR+ research similarly used Camera2 RAW specifically to avoid unwanted ISP tone mapping and spatial denoising. ⁵

Your goal	What “Camera” can realistically guarantee
Force unsupported native 4K/60/120 fps	No. Native stream dimensions and frame rates are bounded by advertised stream configurations and tested session combinations. ⁶
Always provide “4K” anyway	Yes, as 4K Enhanced : capture the highest supported native stream and super-resolve it afterward. It must not be presented as native sensor 4K.
Always provide higher fps anyway	Yes, as FPS Enhanced using motion-compensated interpolation. Interpolated frames are synthesized frames, not extra camera exposures. Frame interpolation is a recognized temporal upsampling technique. ⁷
Universal RAW video	Not at arbitrary resolutions/frame rates. RAW is an output capability and usable RAW stream rates depend on the device's stream map and session support. ⁶
Own photo processing	Largely yes: acquire RAW and perform your own registration, merging, denoise, HDR, super-resolution, color and tone processing. ⁸
Own video image processing	Yes when starting from RAW/YUV/P010, with your own processing before encoding; the HAL/API and ultimately an encoder still necessarily participate.
Preserve a “real” RAW while applying saturation/sharpening/HDR sliders	Yes if the master RAW is immutable and those controls are stored as a render recipe/sidecar rather than destructively baked into the sensor master.
Beat iPhone/Pixel/Galaxy on every device	Impossible to promise universally because sensor, lens, OIS, readout bandwidth, HAL exposure, thermal limits and calibration vary drastically. You can build a pipeline competitive on well-supported hardware and validate this empirically.

Android 17, released in June 2026, materially improves the high-end end of this design by adding `ImageFormat.RAW14`, a densely packed 14-bit sensor format for compatible cameras. Existing devices can still expose `RAW_SENSOR`, RAW10 or RAW12 depending on their capabilities. ⁹

One subtle issue is important: the current `DngCreator` documentation still describes `writeImage()` as accepting `RAW_SENSOR`, while its byte-buffer path expects a 16-bit-per-pixel Bayer representation. Therefore the internal RAW engine should support RAW10/12/14 unpacking into a normalized 16-bit Bayer working buffer, and a dedicated standards-compliant DNG writer should eventually be implemented for formats/metadata that `DngCreator` cannot represent cleanly. ¹⁰

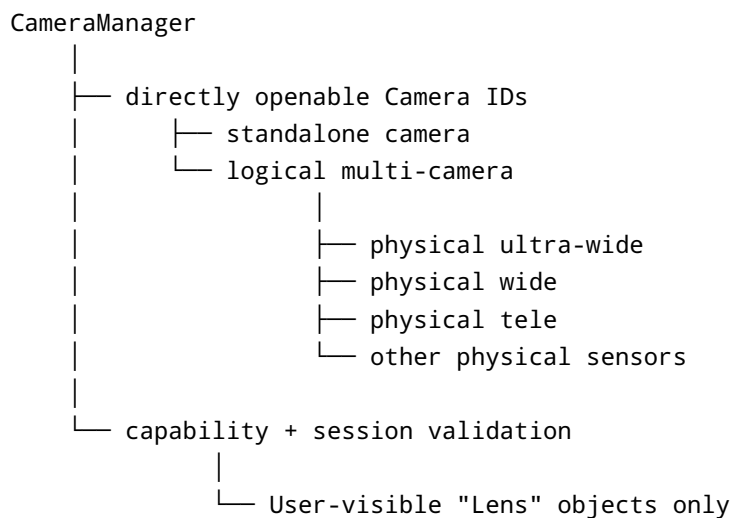
I would **not base this project on leaked Apple, Samsung, Pixel or vendor camera source**. It is unnecessary for the core algorithms, makes licensing and redistribution dangerous, and locks you to implementation details that change between devices. Google's published HDR+, HDR+ Bracketing, Super Res Zoom and Night photography research already gives unusually detailed public descriptions of the most important computational-photography principles. ¹¹

Universal lens discovery and the per-lens capability system

The camera-selection layer should be one of the most carefully engineered parts of the application. **Do not display** `CameraManager.getCameraIdList()` **directly**. Android explicitly states that this list contains directly openable camera devices but may omit physical cameras that are only usable underneath a logical multicamera device. Logical cameras expose their constituent physical IDs separately, and Android provides physical-stream routing through session `OutputConfiguration`s.

12

The discovery graph should therefore look conceptually like this:



Android's logical multicamera system was deliberately designed so an application can open the logical CameraDevice once and route individual output configurations to physical camera IDs. The framework can also expose separate physical result metadata and calibration information; whether particular physical-stream combinations actually work remains dependent on the HAL and must be queried/ tested.

13

Your internal model should not call these objects merely camera IDs. Create something like:

```
LensProfile
├─ persistentLensKey
├─ logicalCameraId
├─ physicalCameraId?
├─ facing
├─ userVisible
├─ userName
├─ userOrder
├─ focalLengths[]
├─ sensorSize
├─ nativeZoomEquivalent
├─ colorFilterArrangement
├─ rawFormats[]
├─ rawResolutions[]
```

- └ previewResolutions[]
- └ videoResolutionsAndFps[]
- └ highSpeedModes[]
- └ manualSensor
- └ manualPostProcessing
- └ rawCapability
- └ burstCapability
- └ tenBitProfiles[]
- └ synchronizationType
- └ intrinsicCalibration
- └ distortionCalibration
- └ maxResolutionPixelMode
- └ verifiedSessionProfiles[]
- └ perLensUserSettings

Camera2 advertises relevant capabilities including RAW, manual sensor, manual post-processing, burst capture, logical multicamera, ten-bit dynamic range, ultra-high-resolution sensors, offline processing and other functionality. Android also distinguishes approximate versus calibrated sensor synchronization for logical multicamera devices. ⁴

Usable-camera filtering should happen in several passes rather than through assumptions about ID numbers:

Probe	Decision
Is it a directly openable logical/standalone camera or an exposed physical member?	Otherwise reject.
Is it a privileged/system camera?	Hide; a normal app cannot make a system-only camera universally accessible. ¹⁴
Can it supply a preview-compatible stream?	Otherwise hide from normal photography UI.
Can a physical stream be routed through its logical parent?	Verify rather than assume. Android supports mapping an <code>OutputConfiguration</code> to a physical camera ID. ¹⁵
Does the required photo session work?	Mark RAW, computational, or processed-only capability separately.
Does it provide a meaningfully different field of view?	If not, merge it into the same logical lens entry unless Developer Cameras is enabled.
Is it depth/IR/monochrome-only?	Do not show as an ordinary color lens; expose it only as an auxiliary input when useful.
Can it be used simultaneously with another physical stream?	Record that capability for seamless lens transitions/depth fusion, but do not require it.
Does a requested resolution/fps combination actually configure?	Only advertise it as Native after validation.

Starting with newer Camera2 APIs, applications can ask whether a proposed session configuration is supported instead of discovering every invalid combination only after trying to start a session. Android's NDK documentation similarly warns that logical/physical stream combinations are not automatically guaranteed and recommends session-configuration support queries. ¹⁶

Cache this result in a **Device Capability Database** keyed to the device/OS/camera-characteristics fingerprint. Re-probe after an OTA or camera-provider change. This means the app may discover, for example, that one phone provides:

Rear :			
Ultra Wide	RAW 12 MP	30 fps video	verified
Main	RAW 50 MP	4K60	verified
Tele 3x	RAW 10 MP	4K30	verified
Front:			
Selfie	no RAW	4K30	verified

while another device legitimately exposes only Main + Selfie. That difference is preferable to showing broken “Camera 2 / Camera 3 / Camera 7” entries.

Per-lens configuration should then be first-class. The user should be able to hide the telephoto, rename “Ultra Wide” to “0.6x”, move 5x before 3x, choose different default resolutions, set distinct HDR strength, denoising, sharpening, saturation, highlight rolloff, shadow lift, color profile, super-resolution strength and video options for every lens. These are application configuration choices and do not require all lenses to expose identical hardware capabilities.

For zoom, use two strategies.

The universal path should use `CONTROL_ZOOM_RATIO` on logical multicamera devices where it behaves well. Android explicitly recommends `CONTROL_ZOOM_RATIO` from API 30 onward for logical multicamera zoom, including optical/digital combinations and zoom-out below 1x. ¹⁷

The advanced path should control physical streams itself when the device supports the required sessions. This enables more predictable lens identity, your own exposure/color matching and custom transition behavior. Android confirms that individual physical streams can be routed and even that RAW buffers from physical cameras can be streamed on supporting multicamera devices. ¹³

For the exact iPhone-like behavior you described, implement a **virtual zoom rail with hysteresis**:

0.60x	-----	0.90x		1.00x	-----	3.0x
		Ultra Wide				Main / Tele
			↑			
			crossover			

Do not hard-code the same optical crossover for every handset. Derive equivalent field of view from focal length and sensor dimensions, then allow a device profile to override it. Your requested default could switch from ultrawide to main once zoom exceeds approximately `0.90x`, while using a slightly lower reverse threshold—for example `0.86x`—to stop the lens from oscillating when the user's finger

sits on the crossover. That threshold is an engineering tuning recommendation rather than an Android platform behavior.

Where two physical streams can coexist, keep both warm around the crossover, normalize exposure/WB/color, geometrically register them using camera intrinsic/extrinsic calibration, then perform a short visual crossfade while the zoom UI animates. Android logical multicamera metadata can provide lens pose, intrinsic and distortion information when the manufacturer exposes it, and sync quality may be advertised as approximate or calibrated. ¹⁸

Where simultaneous physical streams are unavailable, use the logical camera's HAL-managed `CONTROL_ZOOM_RATIO` path or perform a masked session switch. **A seamless optical transition cannot be universally guaranteed by a third-party application**, because physical stream combinations and switching behavior are HAL-dependent. ¹⁹

Computational RAW photo engine

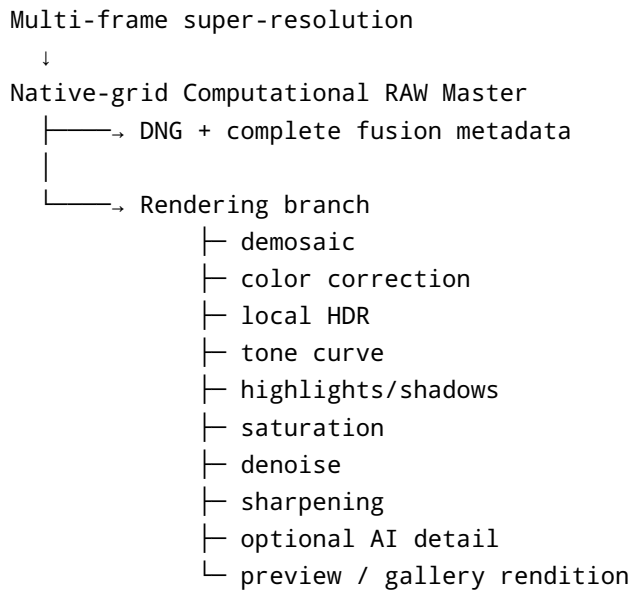
This is the part that can make Camera genuinely different.

Google's published HDR+ architecture validates the core concept behind your design: take multiple Bayer RAW frames, align and merge them to improve noise and dynamic range, rather than starting from already processed JPEG/YUV output. Google's original HDR+ work deliberately started from Bayer RAW to retain more data and avoid ISP tone mapping and spatial denoising; it used burst alignment and robust temporal filtering. ⁵

Later HDR+ Bracketing work added intelligently chosen longer exposures to improve shadow quality while still dealing with moving scenes and ghosting. Google describes the exposure decision as depending on scene dynamic range and motion and performs the merge early in the RAW pipeline so that a computational RAW result remains possible. ²⁰

That suggests your main still pipeline should be:

```
Shutter
  ↓
Scene Analysis + Motion Analysis
  ↓
Adaptive Exposure Scheduler
  ↓
RAW burst + CaptureResult + Gyro/OIS timestamps
  ↓
Black-level / defect / lens-shading normalization
  ↓
Global registration
  ↓
Local tile / subpixel registration
  ↓
Blur + motion + occlusion confidence maps
  ↓
Robust RAW HDR fusion
  ↓
```



Do not destructively convert the master RAW to JPEG or HEIF. Android's DNG system exists precisely to preserve raw sensor-derived pixels and the metadata required to interpret them later. A small rendered thumbnail inside a DNG or app cache is fine; it does not change the master capture into a JPEG. ³

Exposure and frame-count policy

The application should decide burst length from measured conditions rather than simply “night = 20 images.” Google has demonstrated both ends of this concept: ordinary HDR+ historically used bursts of short exposures, whereas experimental very-low-light imaging accumulated dozens of RAW frames under sufficiently static conditions; HDR+ Bracketing later chose exposure combinations according to scene dynamic range and motion. ²¹

A good **initial tuning policy** would be:

Scene	Starting capture policy
Bright daylight, low motion	3–5 short RAW frames
Bright, very high dynamic range	5–8 protected-highlight frames + possibly one longer shadow frame
Indoor	6–10 frames with moderate exposure variation
Handheld night	10–18 short/base frames plus 1–3 longer frames when motion confidence permits
Moving child/pet/action	3–6 faster exposures; prioritize sharp reference imagery
Stable tripod night	16–32 frames, with much longer exposures if sensor limits allow
Astrophotography-style tripod	Separate long-exposure scheduler with star/sky-motion handling

These numbers are proposed starting points, not universal optimal constants. The scheduler should dynamically stop adding frames once estimated SNR gain becomes smaller than motion/thermal/latency costs.

Inputs to that scheduler should include preview histogram clipping, estimated scene illumination, current ISO, exposure time, sensor noise profile when available, gyro angular velocity, OIS state, optical-flow motion, faces/subjects, autofocus confidence, thermal headroom and the user's selected latency/quality preference.

Your three top-level choices can map cleanly to:

Normal favors speed. It can still quietly capture a small constant-exposure RAW burst when conditions benefit from it.

HDR forces a computational burst and stronger dynamic-range protection, but uses predictable parameters.

HDR+ Auto lets the scheduler decide frame count, highlight-protecting exposure, longer shadow exposures, deghost strength and super-resolution according to scene analysis. This closely follows the high-level philosophy disclosed in Google's HDR+ and HDR+ Bracketing research without copying proprietary production code. ²²

Sharpness and blur rejection

Do **not** simply score each image as “blurred/not blurred” and throw away the whole frame.

A long exposure can be soft in a moving person's hands while still contain extremely useful low-noise information in a static wall or dark sky. Your fusion engine should therefore produce **per-tile confidence maps**.

For every frame calculate:

```
confidence(x,y) =  
    registrationConfidence  
    × sharpnessConfidence  
    × motionConsistency  
    × saturationValidity  
    × exposureQuality  
    × noiseWeight  
    × occlusionConfidence
```

Use gyro motion and a frequency/gradient sharpness metric for global screening. Then use local optical flow/phase correlation to determine whether individual tiles agree with the reference. Completely reject frames only for catastrophic shake, failed capture, extreme misfocus or unusable registration.

Google's multiframe super-resolution research explicitly targets robustness to local motion, occlusion and scene changes rather than assuming every pixel in every frame is valid. ²³

Registration and fusion

A high-quality implementation should use a coarse-to-fine hierarchy:

```
Gyroscope prediction
↓
global transform
↓
RAW-aware image pyramid
↓
tile phase correlation / optical flow
↓
sub-pixel refinement
↓
rolling-shutter-aware local warps
↓
occlusion + motion masks
```

The original HDR+ work used FFT-based alignment and a hybrid 2D/3D Wiener technique; you do not have to reproduce that exact implementation, but its published results strongly support combining robust alignment with temporal frequency-domain/noise-aware fusion. ⁵

Model RAW noise as at least a signal-dependent shot-noise term plus signal-independent read noise. HDR+ Bracketing specifically discusses the importance of different exposure lengths for improving low-light shadow SNR while retaining highlights. ²⁰

Fusion should happen in a linearized Bayer/radiance domain. A practical sequence is:

```
raw DN
↓ subtract per-channel black level
normalized photoelectrons/radiance estimate
↓ exposure/ISO normalization
registered samples
↓ robust temporal outlier rejection
HDR weighted fusion
↓
high-SNR Bayer estimate
```

Then apply **multi-frame super-resolution before conventional demosaic wherever the capture provides useful subpixel offsets**. Google's published Super Res Zoom work directly reconstructs complete color values from multiple CFA RAW frames and uses natural hand tremor as a source of subpixel sampling; it simultaneously improves spatial resolution and SNR. ²³

That is also the right answer to your request for **“real photo upscaling.”** Multi-frame super-resolution is preferable to making a generative neural network invent texture because it can extract additional information from genuinely observed, shifted sensor samples. ²³

Two master formats are better than one

Trying to call every enhanced result “RAW” creates a semantic problem. Use two representations:

Camera Master DNG should remain on the original sensor/CFA grid. Merge and denoise multiple exposures into that grid but do not geometrically upscale it. This is the archival, reversible, computational RAW master.

Enhanced Linear Master can contain the super-resolved/demosaiced high-bit-depth result after transformations that no longer make sense as the original Bayer mosaic. This can be represented through an appropriate high-bit-depth linear format/DNG implementation, while never pretending it is untouched sensor mosaic data.

That distinction matters particularly for panorama, perspective correction and aggressive multiframe super-resolution, because those operations geometrically resample pixels and therefore cannot honestly preserve the original sensor CFA geometry.

Metadata

For an ordinary single-frame DNG, Android can derive metadata such as exposure settings from the associated CaptureResult. ³

For an eight-frame computational DNG, there is no single completely truthful ISO/shutter value. The file should therefore expose something like:

Displayed capture metadata

Base/reference frame:

ISO 80
1/120 s
f/1.7
6.8 mm
Main camera

Computational capture:

9 RAW frames
Exposure span: 1/480 – 1/30 s
ISO span: 80 – 200
Fusion mode: HDR+ Auto
Super Resolution: On
Processing engine: Camera Fusion v3.2

The standard EXIF/DNG fields can represent the reference exposure, while an embedded XMP/private metadata block or companion JSON records **every** frame's camera ID, timestamp, exposure time, ISO, frame duration, focal length, focus distance, WB gains, crop, gyro association and fusion weight. That keeps the final file technically honest.

Per-lens HDR and editing

Your sliders—HDR, Highlights, Shadows, Saturation, Sharpness, Denoise, Upscale and similar controls—should operate on the **rendering recipe**, not destructively rewrite the computational RAW master.

That design has an excellent precedent in the broader computational-camera world: Google's dual-exposure interface separates overall brightness/highlight behavior from shadow tone mapping, showing why one generic “exposure” control is insufficient for HDR rendering. ²⁴

Store settings per lens:

```
Ultra Wide:
HDR Auto strength    0.80
Highlights           -18
Shadows              +8
Saturation            0.96
Chroma denoise       0.72
Luma denoise         0.55
Sharpen              0.48
Super-resolution     Auto
Texture preservation High

Main:
HDR Auto strength    0.65
Highlights           -12
Shadows              +4
Saturation            1.00
...
```

The preview render should approximate the eventual result at much lower computational cost. Google used ML-assisted Live HDR+ to approximate the eventual HDR+ appearance in the live viewfinder, demonstrating that real-time preview and expensive final processing can be separate pipelines. ²⁵

AI should similarly be separated into **fidelity AI** and **generative AI**. Multi-frame denoise, deblur, semantic tone mapping, depth estimation and observation-constrained super-resolution can participate in the normal pipeline. Generative replacement of eyelashes, text, moon detail, foliage, skin pores or architecture should never modify the archival RAW master and should be visibly identified as a derived “Generative Enhance” rendition.

Video architecture, RAW video and synthetic resolution

Video requires a different engineering philosophy from still photography.

Camera2 supports repeating capture requests and exposes stream sizes, minimum frame durations and high-speed video combinations. That makes a custom video graph possible, but the framework does **not promise arbitrary RAW dimensions at arbitrary frame rates**. RAW availability and throughput remain tied to the camera's advertised formats, dimensions and session combinations. ²⁶

Build three video paths rather than one.

Pipeline	Input	Processing ownership	Purpose
RAW Video	RAW10/12/14/ RAW_SENSOR when genuinely sustainable	Maximum	Cinema/scientific mode on capable hardware
Direct Computational Video	YUV_420_888 or 10-bit P010 where supported	Very high	Normal high-quality custom video
Performance Fallback	PRIVATE/encoder-oriented stream	Lower	Maximum compatibility, sustained FPS and thermal stability

The second path will probably be the **best real-world recording mode**.

RAW video gives maximum sensor-domain freedom but creates enormous bandwidth, memory, thermal and storage pressure. Direct YUV/P010 allows you to run your own temporal denoise, local HDR, color conversion, sharpening, stabilization and tone curve before passing rendered frames into the video encoder. Ten-bit video profiles should only be exposed where the device advertises compatible dynamic-range profiles; Android specifically provides `DynamicRangeProfiles` because combinations of ten-bit profiles/streams are not universally interchangeable. ²⁷

The video graph should be:

```

Camera RAW/YUV/P010
  ↓
timestamp + gyro synchronization
  ↓
rolling-shutter correction
  ↓
motion estimation
  ↓
temporal RAW/YUV denoise
  ↓
multi-exposure/HDR fusion where feasible
  ↓
electronic stabilization warp
  ↓
lens/color normalization
  ↓
demosaic (RAW path)
  ↓
local tone mapping
  ↓
detail/sharpening
  ↓
optional temporal super-resolution
  ↓
working 10/16-bit GPU surface

```

```
↓
HDR/SDR transfer + gamut mapping
↓
MediaCodec / selected encoder
↓
MP4/MKV/etc.
```

This satisfies your requirement that the **image-processing decisions are yours**, while acknowledging that the sensor/HAL and output codec remain unavoidable infrastructure.

For native recording, never ask a handset for an unadvertised 4K60 configuration and just hope it works. Android's stream-configuration APIs exist specifically to report supported dimensions, high-speed sizes/FPS and timing constraints, and newer APIs let the application test entire session combinations. ²⁸

Instead expose:

```
Resolution
  1080p Native
  1440p Native
  4K Native
  4K Enhanced
  8K Enhanced

Frame Rate
  24 Native
  30 Native
  60 Native
  120 Native
  60 Enhanced
  120 Enhanced
```

For example, if a lens only supports 1080p60 but the user chooses **4K Enhanced**, record the highest-quality 1080p60 stream and run temporal/multi-frame SR to produce a 4K output. If it supports 4K30 but the user selects **4K60 Enhanced**, capture native 4K30 and synthesize intermediate frames. The latter must be labelled synthetic because frame interpolation creates images between captured frames rather than making the sensor expose twice as often. ⁷

That is a much stronger product design than displaying fake “native” modes that fail on half of Android hardware.

For **Slow Motion**, preference order should be:

```
native 240 fps
  ↓ unavailable
native 120 fps + interpolation
  ↓ unavailable
native 60 fps + stronger interpolation
```

↓
normal 30 fps slow-down only

Again, only modes reported through the high-speed stream configuration should receive a Native badge. ²⁹

For **RAW Video**, define a proprietary internal chunked format rather than creating thousands of tiny DNGs during recording:

```
CAMRAW
├─ stream header
├─ sensor/CFA/calibration block
├─ lens metadata
├─ frame chunk
│   ├── timestamp
│   ├── RAW payload
│   ├── exposure metadata
│   └─ gyro samples
├─ frame chunk
├─ ...
└─ index
```

After recording, Camera can optionally transcode/export to a conventional sequence/container. Chunked writing significantly simplifies sequential I/O and crash recovery compared with thousands of independent files; that format design is an application architecture recommendation rather than an Android requirement.

Video should also maintain **separate profiles per lens**:

```
Ultra Wide Video
  max native: 4K30
  preferred: 4K30
  HDR: Auto
  temporal NR: 0.7
  sharpness: 0.4
  stabilization: Strong

Main Video
  max native: 4K60
  preferred: 4K60
  HDR: On
  temporal NR: 0.5
  sharpness: 0.35

Tele Video
  max native: 4K30
  4K60 Enhanced: enabled
```

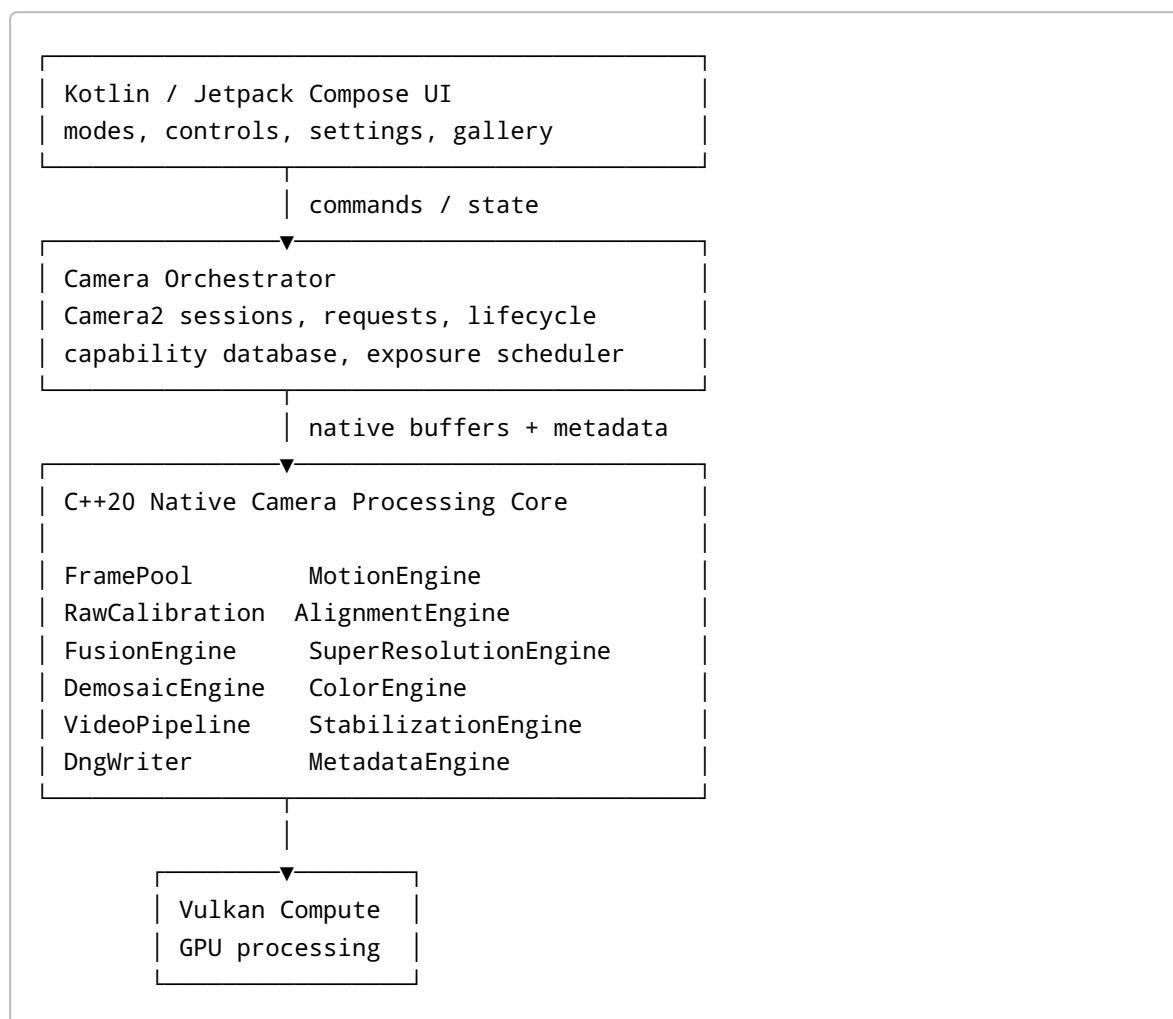
For seamless video lens switching, the app should prefer a logical multicamera stream around transition points because Android's zoom-ratio mechanism is specifically designed to cooperate with logical-camera optical/digital zoom. Explicit physical-stream control can be added when simultaneous physical stream sessions are verified. ³⁰

Native processing architecture and performance

The app should **not** be written entirely in Kotlin, and it also should not be entirely native.

Use Kotlin for Android lifecycle, UI, permissions, settings, MediaStore and orchestration, with C++/ Vulkan responsible for sustained high-bandwidth pixel operations.

A strong architecture is:



`AHardwareBuffer` is particularly valuable because Android explicitly supports zero-copy shared views of hardware buffers and allows them to be bound as Vulkan external memory. That is exactly the type of infrastructure needed to keep preview/video data on GPU-accessible surfaces rather than repeatedly copying multi-megapixel frames through Java heaps. ³¹

A practical division of computation would be:

Work	Preferred processor
Camera/session control	CPU/Kotlin
Metadata parsing	CPU/native
Small RAW unpacking / bookkeeping	CPU C++/NEON
Pyramid generation	Vulkan GPU
Phase correlation	GPU
Dense/tile motion	GPU
Warp/resampling	GPU
Temporal fusion	GPU
Multiframe SR	GPU
Demosaic	GPU
Local tone mapping	GPU
Video temporal filtering	GPU
ML segmentation/depth/detail	GPU/NPU where a suitable runtime/device delegate exists
DNG/container metadata	CPU
Background file writing	Native I/O

Google's original mobile HDR+ implementation itself used Camera2 plus Halide and demonstrated an entirely on-device RAW burst pipeline; later Google research demonstrated full multiframe CFA super-resolution on production mobile hardware. Those historical results establish that this overall class of workload is suitable for mobile devices, although today's implementation should be heavily retuned for current GPUs/NPUs and camera resolutions. ⁸

Internally, avoid doing this:

```

Camera Image
  ↓ memcpy
Java ByteArray
  ↓ memcpy
JNI
  ↓ memcpy
C++ buffer
  ↓ memcpy
GPU texture

```

Aim instead for:

```

Camera HardwareBuffer
  ↓

```



```

AHardwareBuffer
  ↓
Vulkan external memory
  ↓
compute passes
  ↓
encoder/render/storage

```

Android's hardware-buffer API specifically supports Vulkan external-memory access and zero-copy sharing semantics. ³¹

Use a **bounded buffer pool** and explicit backpressure. Never let processing create an unlimited queue of 50 MP RAW frames when a user rapidly presses the shutter.

A suggested memory model is:

```

FramePool
├─ preview ring:      3-5 frames
├─ ZSL RAW ring:     capability-dependent
├─ active burst pool: adaptive
├─ intermediate pyramid buffers: reused
├─ fused RAW buffer: one/two
└─ render buffers:   double/triple buffered

```

For shutter responsiveness, maintain a **zero-shutter-lag-like rolling history** only on devices whose RAW/burst configuration can sustain it. Where continuous RAW is too expensive, keep a fast preview/YUV ring and start the RAW burst immediately at shutter.

Processing must be asynchronous from camera operation:

```

Capture Thread
├─ shutter N
├─ shutter N+1
└─ shutter N+2
    │
    ▼
Processing Queue
├─ │
├─ │
└─ │
    ▼   ▼   ▼
Align Merge Render

```

The UI gets an immediate temporary thumbnail from the reference/preview frame. A circular progress ring overlays that thumbnail while the master completes.

For processing that continues when the UI leaves the foreground, use a persistent job record plus an appropriate foreground-processing mechanism. Android's WorkManager documentation supports

long-running user-important work, but also warns that from Android 16 long-running workers can consume JobScheduler quota and explicitly suggests a directly launched foreground service in applicable cases. ³²

So the architecture should be:

```
User actively shoots
  → in-process processing executor

User backgrounds app while processing
  → foreground processing service + notification

App/process gets killed
  → persisted Room queue
  → WorkManager/recovery mechanism resumes safe stages
```

Store intermediate burst data transactionally so an interrupted merge can be resumed or restarted without losing the photo.

The thumbnail progress animation should report actual stages:

- Captured
- Aligning
- Merging RAW
- Finalizing

Avoid fabricated “63%” values unless the processing graph genuinely knows its remaining work.

Modes, controls, UI and gallery behavior

The visual UI can remain Pixel-like and simple while the settings underneath are extremely sophisticated.

The main viewfinder should therefore expose only:

Flash	HDR+	Auto	Settings
VIEWFINDER			
0.6	1x	3	5
Photo	Portrait	Night	Pro

```
| [recent]  ( shutter )  ⌚  |
```

All complexity belongs in contextual panels and per-lens configuration rather than covering the viewfinder with buttons.

Photo uses the adaptive computational RAW system.

Night raises the maximum burst/exposure budget and changes motion/tripod logic. Google's low-light experiments illustrate why more accumulated exposures are advantageous as illumination decreases and why much longer exposures become practical when the phone is stabilized. ³³

Portrait should preserve the same RAW master and generate depth separately. If a device exposes depth/multicamera information, use it; otherwise run monocular depth on a rendered proxy. Multiview and monocular depth are both established approaches to creating photographic depth maps. ³⁴

Pro should expose all actually supported sensor controls: ISO, exposure time, focus distance, exposure compensation, WB behavior, metering, RAW format, burst strategy and resolution. Camera2's manual-sensor/manual-post-processing capabilities identify devices offering the required level of control. ¹⁴

Slow Motion follows native high-speed tables and provides synthetic interpolation as a separate enhanced mode. ³⁵

Panorama captures exposure/WB-locked overlapping imagery, uses gyro/visual alignment, projects the frames into a common space and merges them using exposure-aware multiband/seam processing. Because panorama warps geometry, its final "RAW" should be called a **linear computational master**, not untouched Bayer RAW.

Astro can be a submode of Night with tripod detection, longer exposure windows and star-aware registration.

Action can prioritize shorter exposure over maximum shadow SNR.

Macro should select the lens with the useful close-focus capability rather than blindly switching to ultrawide based on manufacturer conventions.

Time Lapse and **Intervalometer** are natural additions because the same manual request engine already exists.

Video Pro should expose shutter angle/time, ISO, focus, WB lock, bitrate/quality, codec, stabilization, HDR profile, resolution and fps where supported.

For aspect ratios, the user can have:

```
Sensor / Full RAW
4:3
3:2
16:9
1:1
```

18:9
19.5:9
21:9
Custom

But the application should **not crop the RAW master merely because the preview is 16:9**. The RAW should capture the largest useful sensor region. The aspect ratio is a framing mask/render crop stored as metadata/recipe. This means the user can later recover the areas outside the original composition.

Preview resolution and preview fps should be independent from still-output resolution. Continuously pushing the camera's maximum RAW resolution just to generate the viewfinder is wasteful; use a high-quality low-latency preview stream and reserve maximum RAW for capture. Logical multicamera physical streams may impose additional performance costs, and AOSP warns that physical-camera configurations can reduce capture rate because of resource constraints. ³⁶

A low-resolution/low-latency GPU HDR preview can approximate the final image while the full RAW pipeline remains substantially more expensive, paralleling the architecture behind Google's Live HDR+ viewfinder. ²⁴

The app's advanced settings should revolve around a hierarchy:

Global

- └ output destination
- └ save source burst
- └ processing quality
- └ AI policy
- └ memory/thermal policy
- └ gallery behavior

Per Lens

- └ visibility
- └ label
- └ order
- └ crossover point
- └ photo resolution
- └ RAW format preference
- └ HDR mode/strength
- └ highlights
- └ shadows
- └ saturation
- └ sharpness
- └ denoise
- └ super-resolution
- └ color profile
- └ video resolution
- └ video fps
- └ video HDR

```
| bitrate/quality
| stabilization
```

For albums, Android's MediaStore is suitable for normal camera output. On Android 10 and later an app can supply `RELATIVE_PATH` as a destination hint—for example `Pictures/Camera` or another user-selected album—and use `IS_PENDING` while the item is still being written. ³⁷

Therefore the UX you requested can be:

```
Settings → Save to Album
      ↓
    user chooses once
      ↓
  DataStore remembers selection
      ↓
all new Camera output goes there
```

Do not ask for the destination after every photo.

The thumbnail button should first display/open the newest item produced in the currently selected Camera album. Opening the entire user's existing photo library is different: modern Android deliberately restricts access to media created by other apps and increasingly encourages limited/photo-picker access rather than unlimited gallery permission. ³⁸

So the best privacy-respecting behavior is:

```
tap thumbnail
  → newest Camera photo

swipe/up / Gallery button
  → in-app MediaStore gallery where permission allows
    or system Photo Picker for broader media

long press thumbnail
  → choose active destination album
```

The destination choice persists; privacy grants for arbitrary existing photos must still respect Android's permission model. ³⁸

Recommended implementation specification

I would build **Camera** around capability tiers rather than pretending every Android handset has identical hardware.

Tier	Required features	What Camera enables
Basic	Camera2 preview + capture	Normal photo/video, manual controls that exist
Computational	RAW capability	Multiframe RAW, HDR, Night, computational DNG
Advanced	Burst + manual sensor + RAW	Full adaptive HDR+, robust Pro mode
Multicamera	Logical multicamera + usable physical streams	User-selectable aux lenses, custom transitions
High-res	Ultra-high-resolution sensor support	Maximum-resolution modes
HDR video	Supported ten-bit dynamic-range profile	Custom 10-bit video processing
RAW14	Android 17 + compatible camera	14-bit capture path
Cinema	Sustainable RAW stream session	RAW video mode

Camera2 advertises capabilities such as RAW, BURST_CAPTURE, MANUAL_SENSOR, MANUAL_POST_PROCESSING, LOGICAL_MULTI_CAMERA and ULTRA_HIGH_RESOLUTION_SENSOR specifically so applications can make this kind of capability-dependent decision. ¹⁴

A sensible Android baseline would be a modern minimum API with progressively richer functionality: physical multicamera routing arrived with the Android 9-era multicamera APIs, zoom-ratio control is available from API 30, ten-bit dynamic-range profiles arrived later, newer session-query APIs improve capability probing, and Android 17/API 37 adds RAW14. ³⁹

The source tree could look like:

```
camera/
├─ app/
│   ├── ui/
│   ├── viewfinder/
│   ├── modes/
│   ├── gallery/
│   └─ settings/
├─ camera-core/
│   ├── DeviceDiscovery.kt
│   ├── CapabilityProbe.kt
│   ├── LensRegistry.kt
│   ├── SessionPlanner.kt
│   ├── CaptureScheduler.kt
│   └─ ZoomController.kt
├─ computational-core/
│   ├── native/
│   └─ raw/
```

```

| | | calibration/
| | | alignment/
| | | motion/
| | | fusion/
| | | hdr/
| | | superres/
| | | demosaic/
| | | color/
| | | sharpen/
| | | denoise/
| | | stabilization/
| | | video/
| | | └ metadata/
| | └ vulkan/
| |   | pyramid.comp
| |   | align.comp
| |   | warp.comp
| |   | merge.comp
| |   | sr.comp
| |   | demosaic.comp
| |   | └ tonemap.comp
| |
| | └ storage/
| |   | DngRepository.kt
| |   | MediaStoreRepository.kt
| |   | ProcessingDatabase.kt
| |   | └ ProcessingService.kt
| |
| | └ benchmark/
| |   | camera-lab/
| |   | fusion-tests/
| |   | golden-images/
| |   | └ device-profiles/

```

The most important internal APIs should be stable interfaces, because you will replace algorithms repeatedly:

```

struct RawFrame {
    BufferHandle pixels;
    RawMetadata metadata;
    MotionSamples motion;
};

struct FusionRequest {
    std::span<const RawFrame> frames;
    FusionParameters params;
};

```

```

struct FusionResult {
    RawImage fusedNativeGrid;
    ConfidenceMap confidence;
    FusionMetadata metadata;
};

class FusionEngine {
public:
    virtual FusionResult merge(const FusionRequest&) = 0;
};

```

This isolates capture from processing and lets you develop HDR Fusion v1, v2 and neural-assisted v3 without rewriting Camera2.

For quality, do not optimize toward “looks sharp when zoomed to 300%.” Establish a repeatable test suite measuring:

RAW correctness: black level, clipped values, defective pixels, color calibration, metadata integrity.

Detail: edge/texture resolution before and after SR.

Noise: luminance/chroma noise at multiple exposures and illumination levels.

Dynamic range: highlight preservation versus usable shadows.

Motion: ghosting on faces, hands, leaves, vehicles, water and display screens.

Skin: hue accuracy, local contrast and oversharpening.

Night: moving people against static backgrounds, bright signage, point lights and deep shadows.

Cross-lens consistency: exposure, white balance, color and framing immediately before/after switching.

Video: dropped frames, motion cadence, stabilization, rolling-shutter artifacts, HDR clipping, encoding quality and A/V sync.

Performance: shutter response, time to proxy thumbnail, time to fused master, memory peak, sustained burst rate, battery and temperature.

Then maintain a **golden-scene suite** captured side-by-side with contemporary Pixel, Galaxy and iPhone flagships. Do not try to duplicate their style exactly; establish measurable targets for noise/detail, highlight clipping, motion artifacts, skin tone, zoom transition and shutter latency. Apple's public ProRAW model is particularly relevant conceptually because it combines RAW information with computational photography while preserving extensive post-capture editing latitude, and Apple's camera APIs have also emphasized deferred processing to preserve capture responsiveness. 40

The development order should be deliberate:

First production milestone: perfect one-camera RAW capture, DNG metadata, manual sensor control, full-resolution preview/capture, reliable MediaStore and device probing. Android's DNG API and RAW capability give you the platform foundation for this. ⁴¹

Second milestone: constant-exposure burst registration, noise-aware fusion and computational DNG. Google's public HDR+ work provides the strongest research baseline. ⁵

Third milestone: adaptive HDR bracketing, local motion masks and Night mode. ²¹

Fourth milestone: multiframe super-resolution directly from Bayer RAW. ²³

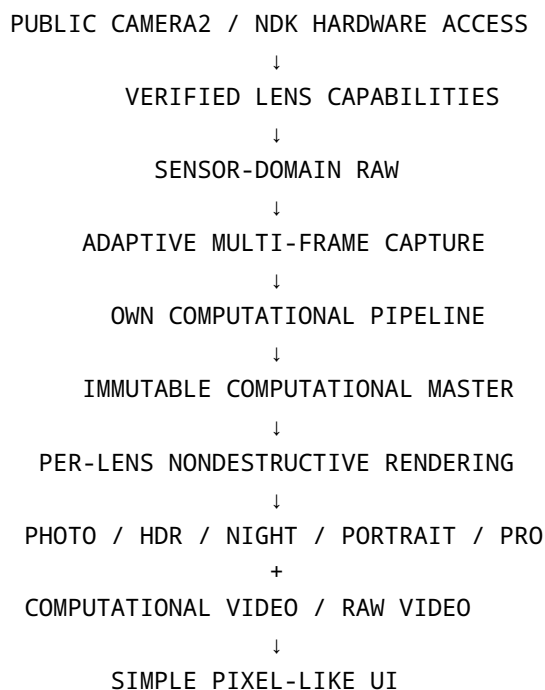
Fifth milestone: physical-lens discovery, per-lens configuration, cross-lens color calibration and seamless zoom rail. Android's logical multicamera and physical-output APIs provide the required platform primitives where the HAL supports them. ¹³

Sixth milestone: custom video pipeline on YUV/P010, native high-speed modes and synthetic-resolution/fps fallbacks. ⁴²

Seventh milestone: RAW video, portrait/depth, panorama, advanced AI restoration and manufacturer/device-specific tuning.

The most important strategic decision is **not** to make the app depend on device-specific hacks from day one. Start with strict public Camera2/NDK capability probing and build optional device profiles only to correct known HAL quirks. Android explicitly acknowledges that complex Camera2 configurations are device-specific, while logical/physical stream behavior depends on individual HAL implementations. ⁴³

If engineered this way, **Camera** is not simply another Camera2 frontend. Its defining architecture becomes:



That design is the closest technically defensible route to what you described: **a universal Android camera that exposes only genuinely usable lenses, keeps RAW as the photographic source of truth, performs its own multiframe HDR/noise reduction/super-resolution, adapts capture depth to available light and motion, provides independent configuration for every lens, offers real versus synthetic resolution/fps transparently, and uses native/GPU processing so sophisticated computational photography does not turn the UI into a slow camera benchmark.** Android's Camera2 multicamera/RAW primitives and the published HDR+/Super Res research demonstrate that the essential pieces exist; the competitive advantage would come from the quality of your capability probing, fusion implementation, calibration, tuning and device validation rather than from hidden or leaked vendor APIs. ⁴⁴

¹ ⁴³ Camera2 overview | Android media

https://developer.android.com/media/camera/camera2?utm_source=chatgpt.com

² ¹³ ¹⁵ ⁴⁴ Multi-camera API | Android media

https://developer.android.com/media/camera/camera2/multi-camera?utm_source=chatgpt.com

³ ⁴¹ DngCreator | API reference

https://developer.android.com/reference/android/hardware/camera2/DngCreator?utm_source=chatgpt.com

⁴ ¹⁴ CaptureResult | API reference | Android Developers

<https://developer.android.com/reference/android/hardware/camera2/CaptureResult>

⁵ ⁸ ¹¹ ²² Burst photography for high dynamic range and low-light ...

https://research.google/pubs/burst-photography-for-high-dynamic-range-and-low-light-imaging-on-mobile-cameras/?utm_source=chatgpt.com

⁶ ¹⁹ Camera | Android NDK

https://developer.android.com/ndk/reference/group/camera?utm_source=chatgpt.com

⁷ Large Motion Frame Interpolation

https://research.google/blog/large-motion-frame-interpolation/?utm_source=chatgpt.com

⁹ ¹⁰ Release notes

https://developer.android.com/about/versions/17/release-notes?utm_source=chatgpt.com

¹² CameraManager | API reference | Android Developers

<https://developer.android.com/reference/android/hardware/camera2/CameraManager>

¹⁶ CameraDevice.CameraDeviceSetup | API reference

https://developer.android.com/reference/android/hardware/camera2/CameraDevice.CameraDeviceSetup?utm_source=chatgpt.com

¹⁷ ³⁰ CaptureRequest | API reference | Android Developers

<https://developer.android.com/reference/android/hardware/camera2/CaptureRequest>

¹⁸ ³⁶ ³⁹ Multi-camera support

https://source.android.com/docs/core/camera/multi-camera?utm_source=chatgpt.com

²⁰ ²¹ HDR+ with Bracketing on Pixel Phones

https://research.google/blog/hdr-with-bracketing-on-pixel-phones/?utm_source=chatgpt.com

²³ Handheld Multi-Frame Super-Resolution

https://research.google/pubs/handheld-multi-frame-super-resolution/?utm_source=chatgpt.com

²⁴ ²⁵ Live HDR+ and Dual Exposure Controls on Pixel 4 and 4a

https://research.google/blog/live-hdr-and-dual-exposure-controls-on-pixel-4-and-4a/?utm_source=chatgpt.com

26 Camera capture sessions and requests | Android media

https://developer.android.com/media/camera/camera2/capture-sessions-requests?utm_source=chatgpt.com

27 42 DynamicRangeProfiles | API reference

https://developer.android.com/reference/android/hardware/camera2/params/DynamicRangeProfiles?utm_source=chatgpt.com

28 29 35 StreamConfigurationMap | API reference

https://developer.android.com/reference/android/hardware/camera2/params/StreamConfigurationMap?utm_source=chatgpt.com

31 Native Hardware Buffer - NDK

https://developer.android.com/ndk/reference/group/a-hardware-buffer?utm_source=chatgpt.com

32 Support for long-running workers | Background work

https://developer.android.com/develop/background-work/background-tasks/persistent/how-to/long-running?utm_source=chatgpt.com

33 Experimental Nighttime Photography with Nexus and Pixel

https://research.google/blog/experimental-nighttime-photography-with-nexus-and-pixel/?utm_source=chatgpt.com

34 The Technology Behind Cinematic Photos

https://research.google/blog/the-technology-behind-cinematic-photos/?utm_source=chatgpt.com

37 38 Access media files from shared storage | App data and files

https://developer.android.com/training/data-storage/shared/media?utm_source=chatgpt.com

40 Capture and process ProRAW images - WWDC21 - Apple Developer

https://developer.apple.com/videos/play/wwdc2021/10160/?utm_source=chatgpt.com